Problem Tracker — Master Code Reference

Every file in the codebase: purpose, notable methods/classes, and which Interview Prep segment (if any) presents it

How to use this: not a presentation script — a map. Skim the index below to find a file fast; the detailed cards underneath explain what's actually worth knowing about it. A blue → NN_MM tag means that file's content is presented live in `Interview Prep/`; no tag means it's real, working code that just isn't part of the 40-minute walkthrough.

Quick Index

File	Lines	Purpose	Interview doc
`tracker.py`	1,025	Playwright scraper; owns the DB schema; syncs NeetCode + LeetCode	03, 04, 06
`dashboard.py`	588	Streamlit app — the 5-tab UI, scoring, recommendations	02, 05
`sm2.py`	96	Canonical SM-2 algorithm, standalone	05
`sm2_vs_heuristic_demo.py`	97	Live comparison script — heuristic vs. SM-2 on real data	05
`send_daily_email.py`	158	Daily 2-review + 1-new email digest	02, 07
`email_config.py`	21	Local-only credential config (gitignored)	02, 07
`com.problemtracker.dailyemail.plist`	—	macOS launchd job, fires the email daily at 7am	07
`migrate_to_unified_db.py`	113	Historical: the two-SQLite-files → one migration	04
`dedupe_leetcode_duplicates.py`	96	Historical: merged 38 slug-alias-gap duplicates	06
`dedupe_query_string_duplicates.py`	109	Historical: this week's 127-row URL-identity bug fix	04
`debug_dom.py`	77	Manual diagnostic — dumps DOM + screenshot from the live browser	—
`test_dashboard.py`	469	46 tests — scoring, queue, recommendations, solved_count integration	08
`test_tracker.py`	98	16 tests — slug parsing, accepted-date extraction	08
`test_sm2.py`	87	13 tests — the SM-2 algorithm + outcome mapping	08
`test_leetcode_sync.py`	682	38 tests — every sync path against a mocked LeetCode context	08
`requirements.txt`	—	Pinned dependencies (playwright, streamlit, pandas)	—
`.gitignore`	—	Protects `email_config.py` + local data from version control	—
`problems.db`	—	The actual SQLite file — schema detailed in segment 3	03

Core Application

`tracker.py` — 1,025 lines

The scraper. Owns the database schema entirely (`init_db()`) and runs standalone via `python3 tracker.py`, driving a CDP-attached Chrome window to sync NeetCode and LeetCode into `problems.db`. Nothing else in the codebase writes to the DB except this file and the dashboard's own review-click path.

- `scrape_neetcode_catalog(page)` — walks the practice page category by category, returns every row (completed or not). Feeds both `catalog` and `problems`. → 03_08
- `SLUG_ALIASES` / `NEETCODE_TO_LEETCODE_SLUG` — 62-entry map reconciling NeetCode's and LeetCode's differently-spelled slugs for the same problem. → 06_05
- `get_leetcode_solved()`, `sync_leetcode_solved()` — LeetCode's own solved-list API cross-check; discovers problems solved entirely outside NeetCode's curriculum. → 06_05
- `extract_all_accepted_dates()`, `fetch_leetcode_accepted_dates()`, `sync_accepted_dates()` — pull *every* distinct accepted date per problem, not just the latest; the data behind the real SM-2 divergence finding. → 03_08, 05_06
- `fetch_leetcode_difficulty_map()`, `sync_difficulties()` — one bulk request gets difficulty for every problem LeetCode knows about, solved or not. → 06_05
- `pick_primary_category()`, `fetch_leetcode_topic_tags()`, `sync_categories()`, `LEETCODE_TAG_TO_CATEGORY` — resolves one category from LeetCode's multi-tag system for problems outside NeetCode's own taxonomy. → 06_05
- `init_db()` — the four-table schema (`problems`, `review_history`, `catalog`, `accepted_dates`), plus `ALTER TABLE` fallbacks for older DBs. → 03_08
- `save_row()`, `save_catalog_row()`, `save_accepted_dates()` — the three upsert functions; none of them ever delete.
- `main()` — orchestrates one full run: scrape → save → sync_leetcode_solved → sync_difficulties → sync_accepted_dates → sync_categories.

`dashboard.py` — 588 lines

The Streamlit app — `streamlit run dashboard.py`. Five tabs (Queue, Recommended, Overdue, Browse, Statistics), reads `problems.db` on every load and writes on every review click.

- `load_problems()` — the central query; combines `accepted_dates` and `review_history` into one `solved_count` via a UNION-based subquery. → 03_08
- `score_row()` — the hand-rolled urgency heuristic: age, starred, last outcome, capped solved-count discount, struggle count. → 05_06
- `build_queue()` — ranks by score, caps per category, backfills if capacity remains. → 05_06, 02_05
- `weak_categories()`, `recommend_new_problems()` — the recommendation engine: coverage-gap categories first, then weakest by struggle rate, easy-to-hard within a category. → 02_05
- `show_dataframe()` — a compatibility wrapper that degrades gracefully on a Streamlit older than the one the venv pins, instead of crashing outright.
- `ensure_tracked()`, `record_review()` — the write path. `ensure_tracked` upserts a bare `problems` row first, so clicking Solved on a never-before-tracked recommendation can't violate `review_history`'s foreign key.
- `render_card()` — the one problem-card widget reused across three of the five tabs.
- `main()` — wires all of the above into the five-tab UI.

Algorithm Comparison

`sm2.py` — 96 lines

A real, tested implementation of canonical SuperMemo-2 — standalone, zero dependency on the rest of the app or the database. Built purely to benchmark against `score_row()`, not to run in production. → 05_06

- `SM2State` (dataclass) — repetitions, interval, ease_factor.
- `sm2_update()`, `replay()` — the algorithm itself, plus a helper folding a chronological quality list into a final state.
- `OUTCOME_TO_QUALITY` — the lossy, explicit mapping from this app's four discrete outcomes onto SM-2's continuous 0–5 scale.

`sm2_vs_heuristic_demo.py` — 97 lines

The script you actually run live in segment 5 — replays every problem's real `accepted_dates` history through `sm2.replay()` and prints it beside the heuristic's score, flagging every case where the two disagree. Read-only, never writes to `problems.db`. → 05_06

- `sm2_next_due()` — one replay per problem, returning the SM-2 state and days-until-due.
- `main()` — prints the full comparison table and the final divergence count (59 of 354, as of the last real run).

Daily Email Digest

`send_daily_email.py` — 158 lines

Emails 2 review picks + 1 new recommendation every morning, by calling the exact same dashboard functions the UI itself uses — no new selection logic, only a delivery layer. → 02_05, 07_05

- `pick_problems()` — `build_queue(df, capacity=2, ...)` for review, `recommend_new_problems(..., per_category=1)` for the new pick.
- `build_email_html()`, `render_problem_html()` — the inline-styled HTML email body.
- `send_email()` — Gmail SMTP via `smtplib`, using an App Password from `email_config.py`.
- `--dry-run` flag on `main()` — prints the picks instead of sending; how this gets tested without touching a real inbox.

`email_config.py` — 21 lines

Local-only config holding the real Gmail App Password — never committed (`.gitignore`). Not algorithmically interesting; worth knowing it exists and why it's separated from `send_daily_email.py` itself (so the credential never has to touch a file that might get shared or diffed).

`com.problemtracker.dailyemail.plist` — launchd job

macOS `launchd` job definition — fires `send_daily_email.py` daily at 7:00 AM via the project's own venv Python. Only fires if the Mac is awake at that moment; no server involved. → 07_05

Historical: One-Time Migrations (kept as evidence, not run anymore)

`migrate_to_unified_db.py` — 113 lines

The original migration — collapses two separate SQLite files (plus a CSV) into one `problems.db` with a real, enforced foreign key. This is the headline story in the "design mistake, fixed" segment. → 04_05

- `load_last_seen_by_url()`, `main()` — reads both legacy DBs, writes the unified schema, never touches or deletes the source files.

`dedupe_leetcode_duplicates.py` — 96 lines

Merged 38 problem pairs that got duplicated because `SLUG_ALIASES` didn't yet cover their NeetCode/LeetCode slug pair — found by asking "are there really no duplicates?" and checking. Thematically tied to the identity-problem story. → 06_05

- `find_duplicate_pairs()` — groups by matching name, case-insensitive.

`dedupe_query_string_duplicates.py` — 109 lines

This week's fix — merged 127 pairs created when NeetCode's `?list=...` query string ended up baked into the primary key. The freshest, most concrete "mistake, fixed" story in the deck. → 04_05

- `pick_survivor()` — prefers the `?list=neetcode150` URL, re-points `review_history` / `accepted_dates` before deleting the loser row.

Diagnostics

`debug_dom.py` — 77 lines

Standalone manual tool — connects to the same live, logged-in CDP browser session and dumps a screenshot plus the real rendered markup. Exists for the day NeetCode changes its UI again (it already has once) and the scraper's selectors stop matching. *Not currently in a presentation segment — good backup material for "how do you debug a scraper that breaks."*

Tests

`test_dashboard.py` — 469 lines · 46 tests

Covers `score_row`, `build_queue`, `weak_categories`, `recommend_new_problems` — plus a real temp-SQLite integration suite (`TestLoadProblemsSolvedCount`) that exercises `load_problems()`'s combined-solved-count SQL directly, not just pre-shaped DataFrames. → 08_03

`test_tracker.py` — 98 lines · 16 tests

Pure-function tests for `leetcode_slug()` and `extract_all_accepted_dates()` / `extract_latest_accepted()` — including the regression test for the "nearest-line, not leftmost-match" fix. → 08_03

`test_sm2.py` — 87 lines · 13 tests

The SM-2 algorithm itself and the outcome→quality mapping — interval growth, the ease-factor floor, failure resets. → 08_03, 05_06

`test_leetcode_sync.py` — 682 lines · 38 tests

The largest suite — every sync path (`sync_leetcode_solved`, `sync_difficulties`, `sync_categories`, `sync_accepted_dates`) against a mocked LeetCode context, no real network calls. → 08_03

Config & Data

`requirements.txt` · `.gitignore` · `problems.db`

Pinned dependencies (`playwright==1.62.0`, `streamlit==1.63.0`, `pandas==3.0.5`); a `.gitignore` protecting `email_config.py`'s real credential and local DB/backup files from ever being committed; and the actual SQLite file itself, whose four-table schema is the subject of segment 3. → 03_08